Data Science — laboratorium

Pandas: Series, DataFrame — część III

Cel zajęć

NumPy nauczył nas pracy z tablicami liczbowymi. W analizie danych bardzo często potrzebujemy jednak czegoś więcej:

- nazw kolumn,
- etykiet wierszy,
- różnych typów danych w jednej tabeli,
- łatwego filtrowania obserwacji,
- obsługi brakujących danych,
- szybkiego obliczania statystyk.

Do takich zadań służy biblioteka **Pandas**.

Podstawowe struktury danych Pandas to:

- `Series` — jednowymiarowa seria danych,
- `DataFrame` — dwuwymiarowa tabela danych.

Każdy przykład wpisuj samodzielnie w **Google Colab**.

1. Importujemy Pandas

Standardowa konwencja:

```
import pandas as pd
import numpy as np
```

Sprawdź wersję:

```
print(pd.__version__)
```

Obserwacja

Tak jak dla NumPy używaliśmy:

```
np
```

tak dla Pandas standardowo używamy:

```
pd
```

2. Pierwszy obiekt Series

Utwórz:

```
scores = pd.Series([72, 88, 61, 95])

print(scores)
```

Sprawdź:

```
type(scores)
```

Obserwacja

Pandas wyświetla dwie rzeczy:

```
0    72
1    88
2    61
3    95
```

Po lewej znajduje się **indeks**, a po prawej wartości.

Sprawdź:

```
scores.index
```

oraz:

```
scores.values
```

Małe zadanie

Utwórz serię:

```
temperatures = pd.Series([18.5, 20.1, 19.8, 23.4, 21.7])
```

Wyświetl:

- całą serię,
- indeks,
- wartości.

3. Własne etykiety indeksu

Indeks nie musi składać się z liczb.

```
scores = pd.Series(
    [72, 88, 61, 95],
    index=["Anna", "Bartek", "Celina", "Daniel"]
)

print(scores)
```

Teraz możemy pobrać wartość za pomocą etykiety:

```
scores["Bartek"]
```

lub kilka wartości:

```
scores[["Anna", "Daniel"]]
```

Obserwacja

W Pandas indeks nie jest tylko numerem pozycji.

Może reprezentować **tożsamość obserwacji**.

Małe zadanie

Utwórz serię zawierającą liczbę godzin nauki:

```
Anna      5
Bartek    8
Celina    3
Daniel   10
```

Następnie wyświetl wynik dla `Celina`.

---

4. Series ze słownika

Serię możemy utworzyć bezpośrednio ze słownika:

```
data = {
    "Anna": 72,
    "Bartek": 88,
    "Celina": 61,
    "Daniel": 95
}

scores = pd.Series(data)

print(scores)
```

To bardzo naturalne połączenie:

```
klucz słownika → indeks
wartość        → wartość Series
```

Sprawdź warunek:

```
scores > 80
```

a następnie:

```
scores[scores > 80]
```

Małe zadanie

Wyświetl tylko osoby, których wynik jest większy lub równy `70`.

---

5. DataFrame — podstawowa tabela Pandas

Teraz utworzymy mały zbiór danych.

```
students = pd.DataFrame({
    "name": [
        "Anna", "Bartek", "Celina", "Daniel",
        "Ewa", "Filip", "Gosia", "Hubert"
    ],
    "group": [
        "KOG", "MAT", "KOG", "MAT",
        "KOG", "MAT", "KOG", "MAT"
    ],
    "hours": [5, 8, 3, 10, 7, 4, 9, 6],
    "score": [72, 88, 61, 95, np.nan, 54, 91, 67],
    "projects": [2, 3, 1, 4, 2, 1, 4, 2]
})

students
```

Obserwacja

`DataFrame` przypomina tabelę:

```
wiersz     → obserwacja
kolumna    → cecha
```

W analizie danych jest to jeden z najważniejszych modeli reprezentacji zbioru danych.

6. Pierwsze oglądanie zbioru danych

Przy dużych tabelach nie chcemy wyświetlać wszystkiego.

Sprawdź:

```
students.head()
```

oraz:

```
students.tail()
```

Możemy określić liczbę wierszy:

```
students.head(3)
```

Sprawdź również:

```
students.shape
students.columns
students.index
students.dtypes
```

Obserwacja

`shape` odpowiada na pytanie:

```
ile wierszy × ile kolumn?
```

`dtypes` pokazuje typ danych każdej kolumny.

Małe zadanie

Bez uruchamiania przewidź:

```
students.shape
```

Dopiero potem sprawdź wynik.

Które kolumny są liczbowe?

7. Wybieramy kolumnę

Jedną kolumnę wybieramy:

```
students["score"]
```

Sprawdź typ:

```
type(students["score"])
```

Obserwacja

Pojedyncza kolumna `DataFrame` jest obiektem:

```
Series
```

Kilka kolumn:

```
students[["name", "score"]]
```

Sprawdź teraz:

```
type(students[["name", "score"]])
```

Obserwacja

Lista nazw kolumn daje nowy:

```
DataFrame
```

Małe zadanie

Wyświetl tylko kolumny:

```
name
hours
score
```

8. Dodajemy nową kolumnę

Do tabeli możemy dopisać nową cechę.

```
students["hours_per_project"] = (
    students["hours"] / students["projects"]
)

students
```

Obserwacja

Nie użyliśmy pętli.

Pandas wykonał działanie na całych kolumnach.

Możemy również utworzyć kolumnę logiczną:

```
students["passed"] = students["score"] >= 60

students
```

Małe zadanie

Dodaj kolumnę:

```
high_score
```

która ma wartość `True`, gdy wynik jest większy lub równy `80`.

9. loc — wybieranie za pomocą etykiet

Operator `loc` służy do wybierania danych na podstawie **etykiet**.

Najpierw ustawmy nazwę studenta jako indeks:

```
students2 = students.set_index("name")

students2
```

Teraz:

```
students2.loc["Anna"]
```

Możemy wskazać również kolumnę:

```
students2.loc["Anna", "score"]
```

Kilka wierszy i kolumn:

```
students2.loc[
    ["Anna", "Daniel", "Gosia"],
    ["group", "hours", "score"]
]
```

Małe zadanie

Za pomocą `loc` wyświetl dla:

```
Bartek
Celina
Hubert
```

kolumny:

```
hours
score
```

10. iloc — wybieranie według pozycji

`iloc` działa podobnie do indeksowania tablic NumPy.

```
students.iloc[0]
```

Pierwsze trzy wiersze:

```
students.iloc[:3]
```

Pierwsze trzy wiersze i pierwsze dwie kolumny:

```
students.iloc[:3, :2]
```

Pojedyncza wartość:

```
students.iloc[1, 3]
```

Zapamiętaj

```
loc  → etykiety
iloc → pozycje
```

Małe zadanie

Za pomocą `iloc` wyświetl:

1. pierwszy wiersz,
2. ostatni wiersz,
3. pierwsze cztery wiersze,
4. pierwsze cztery wiersze i kolumny od `name` do `hours`.

11. Filtrowanie danych

To jedna z najważniejszych operacji w Data Science.

Sprawdź:

```
students["score"] >= 70
```

Otrzymaliśmy maskę logiczną.

Wykorzystaj ją:

```
students[students["score"] >= 70]
```

Małe zadanie

Wyświetl studentów:

1. którzy uczyli się więcej niż `5` godzin,
2. którzy uzyskali wynik większy niż `80`,
3. którzy wykonali co najmniej `3` projekty.

12. Kilka warunków jednocześnie

Chcemy znaleźć osoby, które:

- uczyły się co najmniej `5` godzin,
- uzyskały wynik co najmniej `70`.

```
students[
    (students["hours"] >= 5)
    & (students["score"] >= 70)
]
```

Alternatywa:

```
students[
    (students["score"] >= 90)
    | (students["projects"] >= 4)
]
```

Uwaga

Tak jak w NumPy:

```
&    AND
|    OR
~    NOT
```

Każdy warunek umieszczamy w nawiasach.

Małe zadanie

Znajdź studentów z grupy:

`KOG`

którzy uzyskali wynik co najmniej `70`.

13. loc i filtrowanie razem

Bardzo wygodnym zapisem jest:

```
students.loc[
    students["score"] >= 80,
    ["name", "group", "score"]
]
```

Obserwacja

Jednocześnie:

1. wybieramy wiersze spełniające warunek,
2. wybieramy interesujące nas kolumny.

To bardzo typowy sposób pracy z danymi w Pandas.

Małe zadanie

Wyświetl:

```
name
hours
score
```

dla studentów, którzy uczyli się co najmniej `7` godzin.

14. Brakujące dane

W naszym zbiorze jedna wartość była:

`np.nan`

Sprawdź:

```
students["score"]
```

Znajdź braki:

```
students["score"].isna()
```

Wybierz wiersze z brakującą wartością:

```
students[
    students["score"].isna()
]
```

A teraz policz braki w każdej kolumnie:

```
students.isna().sum()
```

Obserwacja

Brak danych jest w Pandas normalnym elementem pracy ze zbiorem danych.

Nie powinniśmy traktować `NaN` jak zwykłej liczby.

Małe zadanie

Wyświetl wszystkie wiersze, dla których `score` **nie jest brakujący**.

Podpowiedź:

```
notna()
```

15. Usuwanie danych — drop()

Załóżmy, że nie potrzebujemy kolumny `projects`.

```
students.drop(columns="projects")
```

Obserwacja

Sprawdź teraz:

```
students
```

Czy oryginalny `DataFrame` został zmieniony?

Możemy również usuwać wiersze:

```
students.drop(index=0)
```

Małe zadanie

Utwórz nową tabelę:

`simple_students`

z `students`, ale bez kolumn:

`projects`
`hours_per_project`

Nie modyfikuj oryginalnego `students`.

---

16. Reindex — nowy układ indeksu

Wróćmy na chwilę do `Series`.

```
scores = pd.Series(
    [72, 88, 61],
    index=["Anna", "Bartek", "Celina"]
)

scores
```

Teraz:

```
scores.reindex([
    "Celina",
    "Anna",
    "Daniel",
    "Bartek"
])
```

Obserwacja

`Daniel` nie występował w danych.

Pandas utworzył dla niego:

`NaN`

Małe zadanie

Zmień kolejność indeksu na:

`Bartek`
`Anna`
`Celina`

17. Jedna z najważniejszych cech Pandas — wyrównywanie danych

Utwórz:

```
test1 = pd.Series(
    [10, 20, 30],
    index=["Anna", "Bartek", "Celina"]
)

test2 = pd.Series(
    [1, 2, 3],
    index=["Bartek", "Celina", "Daniel"]
)
```

Dodaj:

```
test1 + test2
```

Obserwacja

Pandas nie dodaje danych wyłącznie według pozycji.

Dopasowuje je według **etykiet indeksu**.

Dla:

```
Bartek
Celina
```

istnieją wartości po obu stronach.

Dla:

```
Anna
Daniel
```

jednej wartości brakuje, więc pojawia się `NaN`.

Pytanie

Dlaczego taka własność może być bardzo użyteczna podczas łączenia danych pochodzących z różnych źródeł?

18. Podstawowe statystyki opisowe

Sprawdź:

```
students["score"].mean()
students["score"].min()
students["score"].max()
students["score"].std()
```

Zwróć uwagę, że brakująca wartość nie zablokowała obliczenia.

Małe zadanie

Dla kolumny `hours` znajdź:

- średnią,
- minimum,
- maksimum,
- odchylenie standardowe.

19. Kto miał najlepszy wynik?

Metody:

```
idxmax()
```

i:

```
idxmin()
```

zwracają indeks wartości największej lub najmniejszej.

Sprawdź:

```
students["score"].idxmax()
```

Teraz:

```
students.loc[
    students["score"].idxmax()
]
```

Małe zadanie

Znajdź wiersz studenta/studentki, który poświęcił/ła najwięcej godzin na naukę.

20. describe() — szybki profil danych

Uruchom:

```
students.describe()
```

Obserwacja

Jednym poleceniem otrzymujemy m.in.:

```
count
mean
std
min
25%
50%
75%
max
```

To bardzo wygodny pierwszy krok podczas poznawania nowego zbioru danych.

Sprawdź:

```
students["score"].describe()
```

Małe zadanie

Na podstawie wyniku `describe()` odpowiedz:

1. ile mamy poprawnych wyników `score`,
2. jaki jest wynik średni,
3. jaka jest mediana,
4. jaki jest wynik minimalny,
5. jaki jest wynik maksymalny.

21. Sortowanie danych

Posortuj studentów według wyniku:

```
students.sort_values("score")
```

Od najlepszego:

```
students.sort_values(
    "score",
    ascending=False
)
```

Możemy sortować według kilku kolumn:

```
students.sort_values(
    ["group", "score"],
    ascending=[True, False]
)
```

Małe zadanie

Posortuj studentów:

1. malejąco według liczby godzin,
2. najpierw według grupy, a następnie malejąco według wyniku.

22. Ranking

Pandas może przypisywać wartościom pozycje w rankingu.

```
students["rank"] = students["score"].rank(
    ascending=False
)

students[["name", "score", "rank"]]
```

Obserwacja

Najwyższy wynik otrzymuje rangę:

```
1
```

Małe zadanie

Utwórz ranking studentów na podstawie liczby godzin nauki.

23. unique() — jakie wartości występują?

Sprawdź:

```
students["group"].unique()
```

Otrzymamy różne wartości występujące w kolumnie.

Możemy również policzyć:

```
students["group"].nunique()
```

Małe zadanie

Sprawdź:

- jakie wartości występują w kolumnie `projects`,
- ile różnych wartości występuje.

24. value_counts() — ile razy występuje każda wartość?

Uruchom:

```
students["group"].value_counts()
```

oraz:

```
students["projects"].value_counts()
```

Obserwacja

`value_counts()` jest jedną z najbardziej użytecznych funkcji podczas pierwszego poznawania danych kategorycznych.

Małe zadanie

Utwórz kolumnę:

```
students["passed"] = students["score"] >= 60
```

Następnie użyj:

```
value_counts()
```

aby sprawdzić, ilu studentów zaliczyło.

25. isin() — czy wartość należy do zbioru?

Załóżmy, że interesują nas osoby, które wykonały dwa lub cztery projekty.

```
students["projects"].isin([2, 4])
```

Użyj maski:

```
students[
    students["projects"].isin([2, 4])
]
```

Małe zadanie

Wyświetl studentów o liczbie projektów należącej do:

```
1 lub 3
```

26. apply() — zastosowanie funkcji do danych

Utwórzmy prostą funkcję:

```
def spread(column):
    return column.max() - column.min()
```

Wybierzmy kolumny liczbowe:

```
numeric = students[
    ["hours", "score", "projects"]
]
```

Następnie:

```
numeric.apply(spread)
```

Obserwacja

Funkcja została wykonana oddzielnie dla każdej kolumny.

Możemy również zapisać krócej:

```
numeric.apply(
    lambda x: x.max() - x.min()
)
```

Małe zadanie

Za pomocą `apply()` oblicz średnią każdej z trzech kolumn numerycznych.

27. Korelacja — pierwszy krok w stronę analizy zależności

Interesuje nas pytanie:

Czy większa liczba godzin nauki wiąże się z wyższym wynikiem?

Najpierw wybierzmy dwie kolumny:

```
students[
    ["hours", "score"]
]
```

Następnie:

```
students[
    ["hours", "score"]
].corr()
```

Obserwacja

Macierz korelacji pokazuje współczynnik korelacji pomiędzy parami zmiennych.

Możemy również wykonać:

```
students["hours"].corr(
    students["score"]
)
```

Ważne

Korelacja nie oznacza automatycznie związku przyczynowego.

Na naszym małym sztucznym zbiorze nie powinniśmy również wyciągać poważnych wniosków statystycznych.

Chodzi o poznanie mechanizmu.

Małe zadanie

Sprawdź korelację pomiędzy:

```
projects
score
```

a następnie pomiędzy:

```
hours
projects
```

28. Mini-projekt — pierwsza analiza danych w Pandas

Używając obiektu[1]:

```
students
```

odpowiedz samodzielnie na pytania.

A. Poznaj zbiór

1. Ile ma wierszy?
2. Ile ma kolumn?
3. Jakie są nazwy kolumn?
4. Jakie typy danych występują?
5. W których kolumnach są brakujące wartości?

B. Wybierz dane

6. Wyświetl tylko `name`, `group` i `score`.
7. Wyświetl studentów z grupy `MAT`.
8. Wyświetl osoby z wynikiem co najmniej `80`.
9. Wyświetl osoby, które uczyły się co najmniej `7` godzin i uzyskały wynik co najmniej `70`.

C. Podsumuj dane

10. Oblicz średni wynik.
11. Oblicz medianę wyniku.
12. Znajdź najwyższy wynik.
13. Znajdź studenta z najwyższym wynikiem.
14. Oblicz średnią liczbę godzin nauki.

D. Kategorie

15. Ilu studentów należy do każdej grupy?
16. Ile razy występuje każda liczba projektów?
17. Jakie różne liczby projektów występują w tabeli?

[1] Porównaj: 5. DataFrame — podstawowa tabela Pandas

E. Porządkowanie

18. Posortuj studentów malejąco według wyniku.
19. Wyświetl trzy najlepsze wyniki.

Podpowiedź:

```
students.sort_values(
    "score",
    ascending=False
).head(3)
```

F. Zależności

20. Oblicz korelację pomiędzy liczbą godzin nauki a wynikiem.
21. Czy w tym sztucznym zbiorze korelacja jest dodatnia czy ujemna?

29. Zadanie końcowe

Spróbuj napisać kod, który zwróci:

Trzech studentów z najwyższym wynikiem, którzy uczyli się co najmniej pięć godzin.

Wynik powinien zawierać tylko:

```
name
hours
score
```

Spróbuj połączyć:

```
filtrowanie
→ wybór kolumn
→ sortowanie
→ head()
```

Po zakończeniu części III

Powinnaś/powinieneś potrafić:

- utworzyć `Series`,
- utworzyć `DataFrame`,
- rozumieć rolę indeksu,
- używać `head()`, `tail()`, `shape`, `columns` i `dtypes`,
- wybierać pojedyncze i liczne kolumny,
- dodawać nowe kolumny,
- używać `loc` i `iloc`,
- filtrować dane za pomocą warunków logicznych,
- rozpoznawać brakujące wartości,
- usuwać wiersze i kolumny,
- używać `reindex()`,
- rozumieć wyrównywanie danych według etykiet,
- obliczać podstawowe statystyki,
- używać `describe()`,
- sortować dane,
- tworzyć ranking,
- znajdować wartości unikalne,
- używać `value_counts()` i `isin()`,
- stosować `apply()`,
- obliczyć prostą korelację pomiędzy zmiennymi.

NumPy a Pandas — najważniejsza różnica

W NumPy pracowaliśmy przede wszystkim z:

```
wartościami + pozycją w tablicy
```

W Pandas dochodzi jeszcze bardzo ważna warstwa:

```
wartości + etykiety wierszy + nazwy kolumn
```

Dzięki temu możemy pisać:

```
students.loc[
    students["score"] >= 80,
    ["name", "hours", "score"]
]
```

zamiast zastanawiać się, że interesująca nas informacja znajduje się na przykład:

```
w kolumnach 0, 2 i 3
```

To właśnie ta możliwość pracy z **opisanymi, tabelarycznymi danymi** sprawia, że Pandas jest jednym z podstawowych narzędzi analizy danych w Pythonie.